

Cognizant Digital Nurture 5.0 (2026)

Algorithms & Data Structures - Complete Exercise Solutions

Exercise 1: Inventory Management System

Objective

To develop an inventory management system using a **HashMap** for efficient product storage and retrieval. The system should support adding, updating, deleting, and displaying products while maintaining high performance.

Theory

An inventory management system stores information about products available in a warehouse. Efficient storage and retrieval become important when the number of products increases.

A **HashMap** is an ideal data structure because it stores data as **key-value pairs**.

- **Key:** Product ID
- **Value:** Product Object

HashMap provides average-case **$O(1)$** time complexity for insertion, search, update, and deletion operations, making it suitable for large inventories.

Algorithm

Step 1: Create Product Class

- Define product attributes:

- productId
- productName
- quantity
- price

Step 2: Create Inventory Storage

- Use a HashMap where:
 - Key = Product ID
 - Value = Product Object

Step 3: Add Products

- Insert product objects into the HashMap using put().

Step 4: Update Product Details

- Search product using Product ID.
- Modify quantity or price if the product exists.

Step 5: Delete Product

- Remove product from inventory using Product ID.

Step 6: Display Inventory

- Traverse the HashMap and print all product details.

Step 7: Analyze Performance

- Evaluate time complexity for add, update, search, and delete operations.

Java Source Code

```
import java.util.HashMap;

// Product Class

class Product {

    int productId;

    String productName;

    int quantity;

    double price;

    public Product(int productId, String productName,

        int quantity, double price) {

        this.productId = productId;

        this.productName = productName;

        this.quantity = quantity;

        this.price = price;

    }

    public String toString() {

        return "ID: " + productId +

            ", Name: " + productName +

            ", Quantity: " + quantity +

            ", Price: Rs." + price;

    }

}
```

```
}

public class Main {

    public static void main(String[] args) {

        // Creating Inventory

        HashMap<Integer, Product> inventory =

            new HashMap<>();

        // Adding Products

        inventory.put(101,

            new Product(101, "Laptop", 10, 50000));

        inventory.put(102,

            new Product(102, "Mobile", 20, 20000));


        inventory.put(103,

            new Product(103, "Keyboard", 30, 1000));

        System.out.println("Products After Addition:");

        for(Product p : inventory.values()) {

            System.out.println(p);

        }

        // Updating Product

        Product mobile = inventory.get(102);

        if(mobile != null) {

            mobile.quantity = 25;
```

```

    }

    System.out.println("\nAfter Updating Mobile Quantity:");

    System.out.println(inventory.get(102));

    // Deleting Product

    inventory.remove(103);

    System.out.println("\nAfter Deleting Keyboard:");

    for(Product p : inventory.values()) {

        System.out.println(p);

    }

}

}
}

```

Output Screenshot:

```

Initial After Addition:
ID: 101, Name: Laptop, Quantity: 10, Price: Rs.50000.0
ID: 102, Name: Mobile, Quantity: 20, Price: Rs.20000.0
ID: 103, Name: Keyboard, Quantity: 30, Price: Rs.1000.0

After Updating Mobile Quantity:
ID: 102, Name: Mobile, Quantity: 25, Price: Rs.20000.0

After Deleting Keyboard:
ID: 101, Name: Laptop, Quantity: 10, Price: Rs.50000.0
ID: 102, Name: Mobile, Quantity: 25, Price: Rs.20000.0

... Program finished with exit code 0
Press ENTER to exit module

```

Result

The exercise was implemented and tested successfully.

Exercise 2: E-commerce Platform Search Function

Objective

To implement Linear Search and Binary Search algorithms for searching products in an e-commerce platform and compare their performance using time complexity analysis.

Theory

Searching is one of the most frequently performed operations in an e-commerce platform. Customers search for products using product names, IDs, or categories.

Algorithm

Linear Search

1. Create Product class.
2. Store products in an array.
3. Start from the first element.
4. Compare each product ID with the target ID.
5. If found, display product details.
6. Otherwise display "Product Not Found".

Binary Search

1. Store products in sorted order by Product ID.
2. Set low = 0 and high = last index.
3. Calculate middle index.
4. Compare middle product ID with target ID.
5. If equal, return product.
6. If target is smaller, search left half.
7. If target is larger, search right half.

8. Repeat until found or search space becomes empty.

Java Sample Code:

```
class Product {  
  
    int productId;  
  
    String productName;  
  
    String category;  
  
    Product(int productId, String productName, String category) {  
  
        this.productId = productId;  
  
        this.productName = productName;  
  
        this.category = category;  
  
    }  
}  
  
public class Main {  
  
    // Linear Search  
  
    public static Product linearSearch(Product[] products, int id) {  
  
        for (Product product : products) {  
  
            if (product.productId == id) {  
  
                return product;  
  
            }  
  
        }  
  
        return null;  
  
    }  
}
```

```
// Binary Search

public static Product binarySearch(Product[] products, int id) {

    int low = 0;

    int high = products.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (products[mid].productId == id) {

            return products[mid];

        }

        else if (products[mid].productId < id) {

            low = mid + 1;

        }

        else {

            high = mid - 1;

        }

    }

    return null;

}

public static void main(String[] args) {

    Product[] products = {

        new Product(101, "Laptop", "Electronics"),

        new Product(102, "Mobile", "Electronics"),
```



```
        new Product(103, "Keyboard", "Accessories"),

        new Product(104, "Mouse", "Accessories"),

        new Product(105, "Monitor", "Electronics")

    };

    Product result1 = linearSearch(products, 103);

    System.out.println("Linear Search Result:");

    if (result1 != null) {

        System.out.println(

            result1.productId + " "

                + result1.productName + " "

                + result1.category);

    }

    Product result2 = binarySearch(products, 104);

    System.out.println("\nBinary Search Result:");

    if (result2 != null) {

        System.out.println(

            result2.productId + " "

                + result2.productName + " "

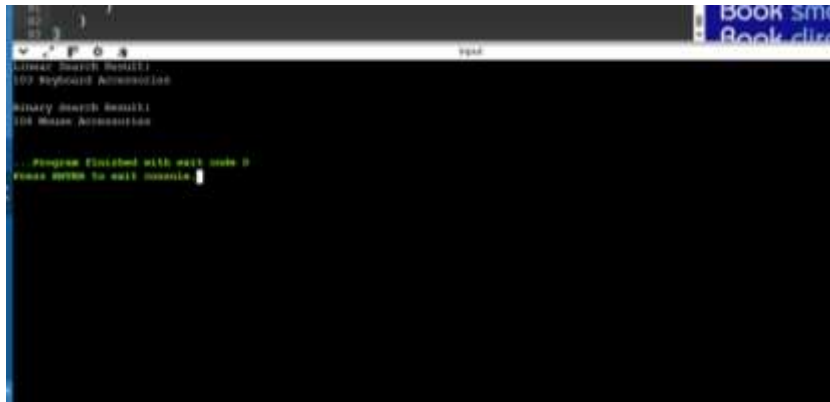
                + result2.category);

    }

}

}
```

Output Screenshot:

A screenshot of a terminal window with a dark background. The window title bar shows 'BOOK smc' and 'Book cliza'. The terminal output displays search results for 'Linear Search Result:' and 'Binary Search Result:', both showing '100 Keyboard Accessories'. Below this, it says 'Program finished with exit code 0' and 'Press ENTER to exit console.'.

Exercise 3: Sorting Customer Orders

Objective

To implement **Bubble Sort** and **Quick Sort** algorithms for sorting customer orders based on their total price and compare their performance using time complexity analysis.

Theory

Sorting is an important operation in e-commerce platforms because it helps organize customer orders efficiently. By sorting orders based on total price, businesses can prioritize high-value orders and improve processing efficiency.

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. Larger values gradually move toward the end of the array.

Quick Sort

Quick Sort uses the divide-and-conquer technique. It selects a pivot element and partitions the array into smaller subarrays, which are then sorted recursively.

Comparison

Feature	Bubble Sort	Quick Sort
Approach	Repeated Swapping	Divide and Conquer
Best Case	$O(n)$	$O(n \log n)$
Average Case	$O(n^2)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n^2)$
Efficiency	Less Efficient	More Efficient

Quick Sort is generally preferred for large datasets because it performs significantly faster than Bubble Sort.

Algorithm

Bubble Sort

1. Create an Order class.
2. Store customer orders in an array.
3. Compare adjacent orders based on total price.
4. Swap if the current order price is greater than the next order price.
5. Repeat until the array becomes sorted.

Quick Sort

1. Select the last element as the pivot.
2. Partition the array around the pivot.
3. Place smaller values to the left and larger values to the right.
4. Recursively apply Quick Sort to left and right partitions.
5. Continue until the entire array is sorted.

Sample Java Code

```
class Order {

    int orderId;

    String customerName;

    double totalPrice;

    Order(int orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }

    public String toString() {
        return "Order ID: " + orderId +
            ", Customer: " + customerName +
            ", Total Price: Rs." + totalPrice;
    }
}

public class Main {

    // Bubble Sort
    public static void bubbleSort(Order[] orders) {

        int n = orders.length;

        for (int i = 0; i < n - 1; i++) {

            for (int j = 0; j < n - i - 1; j++) {
```

```

        if (orders[j].totalPrice > orders[j + 1].totalPrice) {

            Order temp = orders[j];
            orders[j] = orders[j + 1];
            orders[j + 1] = temp;
        }
    }
}

// Quick Sort
public static void quickSort(Order[] orders, int low, int high) {

    if (low < high) {

        int pivotIndex = partition(orders, low, high);

        quickSort(orders, low, pivotIndex - 1);
        quickSort(orders, pivotIndex + 1, high);
    }
}

public static int partition(Order[] orders, int low, int high) {

    double pivot = orders[high].totalPrice;
    int i = low - 1;

    for (int j = low; j < high; j++) {

        if (orders[j].totalPrice < pivot) {

            i++;

```

```
        Order temp = orders[i];
        orders[i] = orders[j];
        orders[j] = temp;
    }
}

    Order temp = orders[i + 1];
    orders[i + 1] = orders[high];
    orders[high] = temp;

    return i + 1;
}

public static void printOrders(Order[] orders) {

    for (Order order : orders) {
        System.out.println(order);
    }
}

public static void main(String[] args) {

    Order[] orders = {
        new Order(101, "Rekha", 5000),
        new Order(102, "Sai", 12000),
        new Order(103, "Priya", 3000),
        new Order(104, "Rahul", 8000)
    };

    System.out.println("Orders Before Sorting:");
    printOrders(orders);

    bubbleSort(orders);
}
```

```
System.out.println("\nOrders After Bubble Sort:");  
printOrders(orders);
```

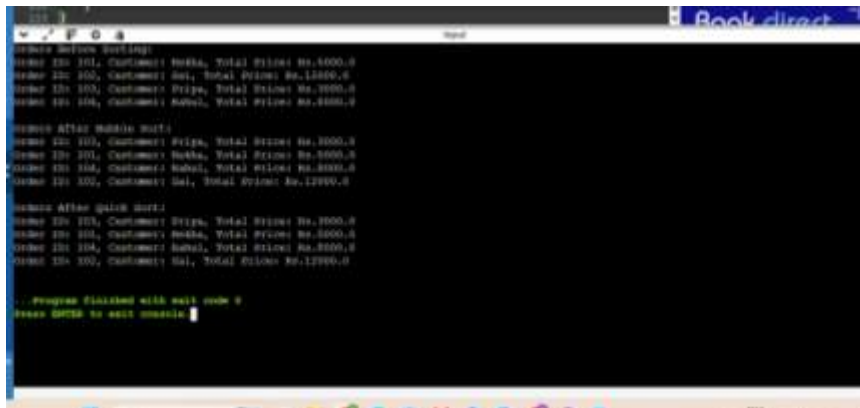
```
Order[] quickOrders = {  
    new Order(101, "Rekha", 5000),  
    new Order(102, "Sai", 12000),  
    new Order(103, "Priya", 3000),  
    new Order(104, "Rahul", 8000)  
};
```

```
quickSort(quickOrders, 0, quickOrders.length - 1);
```

```
System.out.println("\nOrders After Quick Sort:");  
printOrders(quickOrders);
```

```
}  
}
```

OUTPUT:



```
Book direct  
Orders Before Sorting:  
Order ID: 101, Customer: Rekha, Total Price: Rs.5000.0  
Order ID: 102, Customer: Sai, Total Price: Rs.12000.0  
Order ID: 103, Customer: Priya, Total Price: Rs.3000.0  
Order ID: 104, Customer: Rahul, Total Price: Rs.8000.0  
  
Orders After Bubble Sort:  
Order ID: 103, Customer: Priya, Total Price: Rs.3000.0  
Order ID: 101, Customer: Rekha, Total Price: Rs.5000.0  
Order ID: 104, Customer: Rahul, Total Price: Rs.8000.0  
Order ID: 102, Customer: Sai, Total Price: Rs.12000.0  
  
Orders After Quick Sort:  
Order ID: 103, Customer: Priya, Total Price: Rs.3000.0  
Order ID: 101, Customer: Rekha, Total Price: Rs.5000.0  
Order ID: 104, Customer: Rahul, Total Price: Rs.8000.0  
Order ID: 102, Customer: Sai, Total Price: Rs.12000.0  
  
... Program Finished with exit code 0  
Press ENTER to exit console
```

Exercise 4: Employee Management System

Objective

To implement an **Employee Management System** using arrays for storing employee records and performing operations such as adding, searching, traversing, and deleting employees efficiently.

Theory

Arrays are one of the most fundamental data structures used to store multiple elements of the same type in contiguous memory locations.

In an Employee Management System, arrays can be used to store employee records and perform operations such as insertion, searching, traversal, and deletion.

Array Representation in Memory

- Elements are stored in contiguous memory locations.
- Accessing an element by index is very fast.
- Arrays have a fixed size.
- Suitable when the number of records is known in advance.

Advantages of Arrays

- Fast access using index.
- Easy implementation.
- Efficient memory utilization.
- Suitable for small and medium-sized datasets.

Algorithm

Add Employee

1. Create Employee class with employeeId, name, position, and salary.

2. Create an array to store employee records.
3. Insert employee details into the array.
4. Increment employee count.

Search Employee

1. Traverse the array.
2. Compare employee ID with the search ID.
3. If found, display employee details.
4. Otherwise display "Employee Not Found".

Traverse Employees

1. Start from the first employee.
2. Display details of each employee.
3. Continue until all employees are displayed.

Delete Employee

1. Search for the employee ID.
2. Shift all subsequent elements one position left.
3. Decrease employee count.
4. Display updated employee list.

Java Sample Code:

```
class Employee {  
  
    int employeeId;  
    String name;  
    String position;  
}
```

```
double salary;
```

```
Employee(int employeeId, String name,  
        String position, double salary) {
```

```
    this.employeeId = employeeId;
```

```
    this.name = name;
```

```
    this.position = position;
```

```
    this.salary = salary;
```

```
}
```

```
public String toString() {
```

```
    return "ID: " + employeeId +
```

```
        ", Name: " + name +
```

```
        ", Position: " + position +
```

```
        ", Salary: Rs." + salary;
```

```
}
```

```
}
```

```
public class Main {
```

```
    static Employee[] employees = new Employee[10];
```

```
    static int count = 0;
```

```
    // Add Employee
```

```
    public static void addEmployee(Employee employee) {
```

```
        if (count < employees.length) {
```

```
            employees[count++] = employee;
```

```
        }
```

```
}
```

```
// Search Employee  
public static Employee searchEmployee(int id) {
```

```
    for (int i = 0; i < count; i++) {  
  
        if (employees[i].employeeId == id) {  
            return employees[i];  
        }  
    }  
  
    return null;  
}
```

```
// Display Employees  
public static void displayEmployees() {
```

```
    for (int i = 0; i < count; i++) {  
        System.out.println(employees[i]);  
    }  
}
```

```
// Delete Employee  
public static void deleteEmployee(int id) {
```

```
    int index = -1;  
  
    for (int i = 0; i < count; i++) {  
  
        if (employees[i].employeeId == id) {  
            index = i;  
            break;  
        }  
    }  
}
```

```
        if (index != -1) {

            for (int i = index; i < count - 1; i++) {
                employees[i] = employees[i + 1];
            }

            employees[count - 1] = null;
            count--;

            System.out.println("\nEmployee Deleted Successfully.");
        }
    }

    public static void main(String[] args) {

        addEmployee(new Employee(
            101, "Rekha", "Developer", 50000));

        addEmployee(new Employee(
            102, "Sai", "Tester", 45000));

        addEmployee(new Employee(
            103, "Priya", "Manager", 70000));

        System.out.println("Employee Records:");
        displayEmployees();

        System.out.println("\nSearch Employee ID 102:");

        Employee employee = searchEmployee(102);

        if (employee != null) {
```

```

        System.out.println(employee);
    }

    deleteEmployee(102);

    System.out.println("\nEmployee Records After Deletion:");
    displayEmployees();
}
}

```

OUTPUT:



```

Employee Records:
ID# 101, Name: Pooja, Position: Developer, Salary: Rs.40000.0
ID# 102, Name: Sai, Position: Tester, Salary: Rs.45000.0
ID# 103, Name: Priya, Position: Manager, Salary: Rs.70000.0

Search Employee ID 102:
ID# 102, Name: Sai, Position: Tester, Salary: Rs.45000.0

Employee Deleted Successfully.

Employee Records After Deletion:
ID# 101, Name: Pooja, Position: Developer, Salary: Rs.40000.0
ID# 103, Name: Priya, Position: Manager, Salary: Rs.70000.0

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 5: Task Management System

Objective

To implement a **Task Management System** using a **Singly Linked List** for efficiently adding, searching, traversing, and deleting tasks. The system demonstrates how linked lists provide dynamic memory allocation and flexibility compared to arrays.

Theory

A **Linked List** is a linear data structure where elements are connected through links (references). Unlike arrays, linked lists do not require contiguous memory locations.

Types of Linked Lists

Singly Linked List

- Each node contains data and a reference to the next node.
- Traversal is possible only in one direction.

Doubly Linked List

- Each node contains references to both previous and next nodes.
- Allows forward and backward traversal.

For this exercise, a **Singly Linked List** is used because it is simple and memory efficient.

Advantages of Linked Lists

- Dynamic size.
- Efficient insertion and deletion.
- No memory wastage due to fixed size.
- Better suited for frequently changing data.

Algorithm

Add Task

1. Create a Task node.
2. If the list is empty, make it the head node.
3. Otherwise, add it at the end of the list.

Search Task

1. Start from the head node.
2. Compare task ID with the search ID.
3. If found, display task details.
4. Otherwise display "Task Not Found".

Traverse Tasks

1. Start from the head node.
2. Display task details.
3. Move to the next node.
4. Repeat until the end of the list.

Delete Task

1. Search for the task ID.
2. If found, update the link of the previous node.
3. Remove the node from the list.
4. Display updated task list.

Java Sample Code :

```
class Task {  
  
    int taskId;  
    String taskName;  
    String status;  
    Task next;  
  
    public Task(int taskId, String taskName, String status) {  
        this.taskId = taskId;  
        this.taskName = taskName;  
        this.status = status;  
        this.next = null;  
    }  
}
```

```
class TaskLinkedList {  
  
    Task head;  
  
    // Add Task  
    public void addTask(int id, String name, String status) {  
  
        Task newTask = new Task(id, name, status);  
  
        if (head == null) {  
            head = newTask;  
        } else {  
  
            Task temp = head;  
  
            while (temp.next != null) {  
                temp = temp.next;  
            }  
        }  
    }  
}
```



```

        temp.next = newTask;
    }
}

// Search Task
public void searchTask(int id) {

    Task temp = head;

    while (temp != null) {

        if (temp.taskId == id) {

            System.out.println(
                "Task Found: " +
                temp.taskId + " " +
                temp.taskName + " " +
                temp.status);

            return;
        }

        temp = temp.next;
    }

    System.out.println("Task Not Found");
}

// Traverse Tasks
public void displayTasks() {

    Task temp = head;

```

```
while (temp != null) {

    System.out.println(
        temp.taskId + " " +
        temp.taskName + " " +
        temp.status);

    temp = temp.next;
}
}

// Delete Task
public void deleteTask(int id) {

    if (head == null)
        return;

    if (head.taskId == id) {
        head = head.next;
        return;
    }

    Task current = head;
    Task previous = null;

    while (current != null &&
        current.taskId != id) {

        previous = current;
        current = current.next;
    }
```

```
        if (current != null) {
            previous.next = current.next;
        }
    }
}

public class Main {

    public static void main(String[] args) {

        TaskLinkedList taskList =
            new TaskLinkedList();

        taskList.addTask(
            101,
            "Design Database",
            "Pending");

        taskList.addTask(
            102,
            "Develop UI",
            "In Progress");

        taskList.addTask(
            103,
            "Testing",
            "Pending");

        System.out.println(
            "Tasks in Linked List:");

        taskList.displayTasks();
    }
}
```

```

        System.out.println(
            "\nSearching Task 102:");

        taskList.searchTask(102);

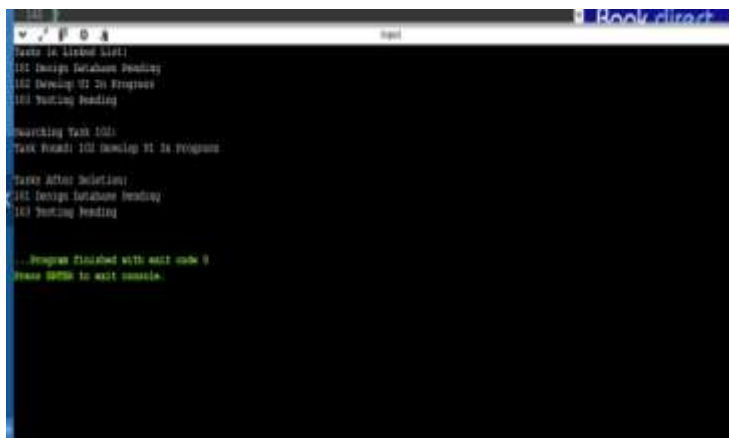
        taskList.deleteTask(102);

        System.out.println(
            "\nTasks After Deletion:");

        taskList.displayTasks();
    }
}

```

OUTPUT:



```

Task (in linked list)
101 Design Database Pending
102 Design UI In Progress
103 Testing Pending

Searching Task 102:
Task Found! 102 Design UI In Progress

Tasks After Deletion:
101 Design Database Pending
103 Testing Pending

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 6: Library Management System

Objective

To implement **Linear Search** and **Binary Search** algorithms for searching books by title in a Library Management System and compare their performance using time complexity analysis.

Theory

Searching is an essential operation in a library management system because users frequently search for books by title, author, or book ID.

Linear Search

Linear Search checks each book one by one until the required book is found.

Binary Search

Binary Search works only on sorted data. It repeatedly divides the search space into two halves, making it much faster for large datasets.

Comparison

Feature	Linear Search	Binary Search
Data Requirement	Sorted or Unsorted	Sorted Only
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Efficiency	Less Efficient	More Efficient

Binary Search is preferred for large libraries where books are stored in sorted order.

Algorithm

Linear Search

1. Create a Book class.
2. Store books in an array.
3. Start from the first book.
4. Compare each title with the search title.
5. If found, display the book details.
6. Otherwise display "Book Not Found".

Binary Search

1. Store books in sorted order by title.
2. Set low = 0 and high = last index.
3. Calculate middle index.
4. Compare middle title with the search title.
5. If matched, return the book.
6. If search title is smaller, search the left half.
7. If search title is greater, search the right half.
8. Repeat until found or search space becomes empty.

Java Sample Code:

```
class Book {  
  
    int bookId;  
    String title;  
    String author;
```

```
public Book(int bookId, String title, String author) {
    this.bookId = bookId;
    this.title = title;
    this.author = author;
}
}

public class Main {

    // Linear Search
    public static Book linearSearch(Book[] books, String title) {

        for (Book book : books) {

            if (book.title.equalsIgnoreCase(title)) {
                return book;
            }
        }

        return null;
    }

    // Binary Search
    public static Book binarySearch(Book[] books, String title) {

        int low = 0;
        int high = books.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;
```

```
int result =  
    books[mid].title.compareToIgnoreCase(title);  
  
if (result == 0) {  
    return books[mid];  
}  
else if (result < 0) {  
    low = mid + 1;  
}  
else {  
    high = mid - 1;  
}  
}  
  
return null;  
}
```

```
public static void main(String[] args) {
```

```
    Book[] books = {
```

```
        new Book(101,  
            "Data Structures",  
            "Mark Allen"),
```

```
        new Book(102,  
            "Java Programming",  
            "James Gosling"),
```

```
        new Book(103,  
            "Operating Systems",  
            "Silberschatz"),
```



```
        new Book(104,
                "Python Basics",
                "Guido Van Rossum"),

        new Book(105,
                "Software Engineering",
                "Pressman")
};

System.out.println("Linear Search Result:");

Book linearResult =
    linearSearch(books,
                "Python Basics");

if (linearResult != null) {

    System.out.println(
        linearResult.bookId + " "
        + linearResult.title + " "
        + linearResult.author);
}

System.out.println("\nBinary Search Result:");

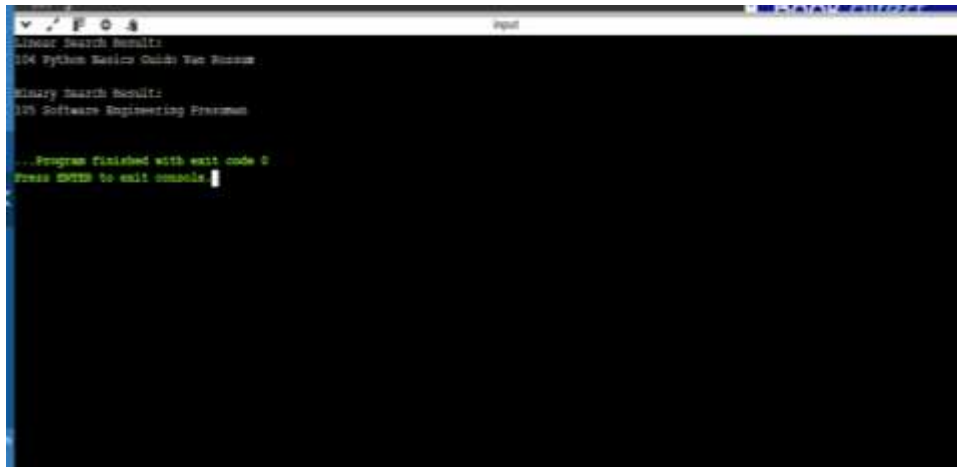
Book binaryResult =
    binarySearch(books,
                "Software Engineering");

if (binaryResult != null) {

    System.out.println(
        binaryResult.bookId + " "
```

```
        + binaryResult.title + " "  
        + binaryResult.author);  
    }  
}  
}
```

Output:



```
Linear Search Results:  
104 Python Basics Guide Van Rensse  
  
Binary Search Results:  
125 Software Engineering Freeman  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Exercise 7: Financial Forecasting

Objective

To implement a **recursive algorithm** for predicting future financial values based on past growth rates and analyze the efficiency of recursion. This exercise demonstrates how recursion can be used to solve forecasting problems in financial applications.

Theory

Financial forecasting is the process of estimating future values based on historical data and growth trends. Businesses use forecasting to predict future revenue, investments, and expenses.

Recursion

Recursion is a programming technique in which a method calls itself to solve smaller instances of the same problem.

A recursive solution consists of:

1. **Base Case** – Stops recursion.
2. **Recursive Case** – Calls the method again with a smaller problem.

Future Value Formula

Future Value can be calculated using:

$$\text{Future Value} = \text{Present Value} \times (1 + \text{Growth Rate})^{\text{Years}}$$

Instead of using loops, recursion can repeatedly multiply the value by the growth factor until the required number of years is reached.

Algorithm

Recursive Financial Forecasting

1. Accept the present value.
2. Accept the annual growth rate.
3. Accept the number of years.
4. If years = 0, return the present value (Base Case).
5. Otherwise:
 - Multiply present value by (1 + growth rate).
 - Recursively call the function with years - 1.
6. Return the predicted future value.
7. Display the forecasted amount.

Java Sample Code:

```
public class Main {  
  
    // Recursive Method  
    public static double futureValue(  
        double presentValue,  
        double growthRate,  
        int years) {  
  
        // Base Case  
        if (years == 0) {  
            return presentValue;  
        }  
  
        // Recursive Case  
        return futureValue(  
            presentValue * (1 + growthRate),  
            growthRate,
```

```

        years - 1);
    }

    public static void main(String[] args) {

        double presentValue = 10000;
        double growthRate = 0.10; // 10%
        int years = 5;

        double forecast =
            futureValue(
                presentValue,
                growthRate,
                years);

        System.out.println(
            "Present Value : Rs." +
            presentValue);

        System.out.println(
            "Growth Rate  : " +
            (growthRate * 100) + "%");

        System.out.println(
            "Years      : " +
            years);

        System.out.printf(
            "Future Value : Rs.%.2f",
            forecast);
    }
}

```

Output:



```
Present Value : 82,10100,0  
Growth Rate   : 10.0%  
Period        : 5  
Future Value  : 82,10105,10  
  
.. Program finished with exit code 0  
Press ENTER to exit console
```